SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section introduces the IndoRoute AI strategic solution, addressing the regulatory compliance hurdles for SMEs in the Malaysian and Indonesian skincare markets as highlighted in the Product Review Documentation.

1.1. Purpose:

The system analysis document highlights the technical scope and design decisions behind the Indoroute AI development.
So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *It's a cloud-native platform deployed entirely on **Vercel**. The services are logically separated into serverless micro-agents (Compliance, Analysis, and Notification) that operate within a serverless environment to ensure high availability and zero-cost scaling.*
2. ***Data Flows:***
    a. *How the raw product ingredient data (from user photo/upload) moves through the FastAPI backend (Vercel Functions), gets processed by the Z.AI GLM reasoning engine, stored in Neon PostgreSQL in a persistent manner, and finally returns a compliance risk report to the user.*
3. ***Model Process:***
    a. *Shows the end-to-end workflow: Uploading a label -> OCR/Extraction -> GLM Compliance Check -> Digital Report Generation.*
4. ***Role of Reference:***
    a. *This serves as a technical blueprint for the UM development team and QA testers to ensure API contracts and logic flows are maintained.*

1.2. Background:

Many local cosmetic vendors struggle with BPOM/NPRA compliance due to manual cross-referencing of banned substances. IndoRoute AI automates this using LLM reasoning.

1. *Previous Version:*
    Existing solutions are manual PDF checklists or static databases that don't account for complex chemical synonyms.
2. *Changes in Major Architectural Components:*
    Transitioned from a static regex-based search to a Semantic LLM Reasoning layer.
3. *New capabilities that are only introduced in this:*
    Real-time compliance "risk scoring" and automated ingredient translation.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
SME Vendors (Skincare)	*Upload ingredient lists or product labels to check for regulatory compliance.*	*High accuracy in detection, fast turnaround time, and clear "Pass/Fail" explanations.*
Regulatory Consultants	*Use the system to audit client products and generate compliance reports.*	*Well-structured digital reports and historical data tracking for audit trails.*
End Consumers	*Access public safety databases to verify product compliance and learn about banned ingredients.*	*Transparency in product safety, easy-to-understand chemical explanations, and real-time verification of local brands.*
Development Team	*Build and maintain the full-stack serverless platform on Vercel.*	*Clear API documentation for GLM integration, modular code, and efficient database indexing.*
QA Team	*Validate the AI's accuracy against known banned substance lists.*	*Robust test cases for edge-case chemicals and high precision in JSON response parsing.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Mobile-Responsive Next.js)*
Architecture	*Serverless Cloud-Native (Vercel)*

Indoroute AI is structured as a client-server system hosted on Vercel. The Next.js Frontend communicates with FastAPI Backend Functions. All logic is processed via independent serverless triggers: Compliance Service and Notification Service. All data is persisted in Neon PostgreSQL (Serverless RDS).

2.1.2 LLM as Service Layer

The architecture integrates Z.AI GLM as the core reasoning engine. It acts as a middleware that transforms unstructured OCR text into structured compliance JSON. The following dependency diagram includes:

2.1.3 Dependency Diagram

a. ***Prompt Construction & Transmission***
Prompts are built in the FastAPI (Vercel Function). The backend fetches raw OCR text and the relevant regulatory guidelines from Neon PostgreSQL. These are injected into a system prompt that defines the GLM's role as a "Regulatory Compliance Officer." The final payload is sent via HTTPS POST to the GLM API.

b. ***Context Window Content***
- **System Role:** Format constraints (Strict JSON).
- **Regulatory Reference:** Banned/restricted substance lists from the DB.
- **User Input:** Extracted ingredient list from the scan.
- **Examples:** 1-2 samples of correct classification to reduce errors.

c. ***Receiving & Parsing Responses***
   i. Verify JSON structure.
   ii. *Extract risk_score, violation_list, and recommendations.*
   iii. *Pass the validated data to the Frontend and the Neon DB.*

© UMHackathon 2026

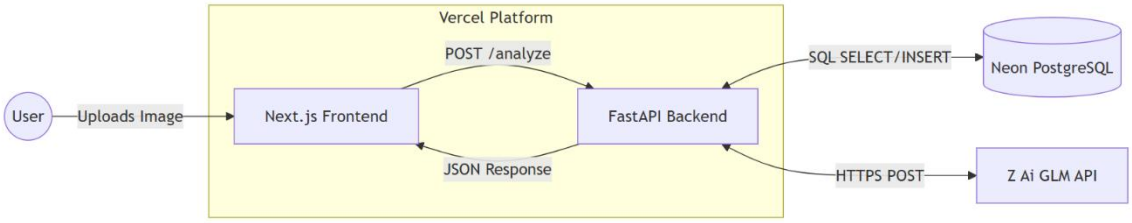
d. Token Enforcement & Chunking

- **Chunking:** Lists over 3,000 tokens are split by ingredient groups using a recursive splitter
- **Stripping:** Non-essential metadata is removed before the API call.
- **Safety:** A per-session counter tracks usage to stop API overages.

e. Major API calls

Source Component	Target Component	Protocol	Purpose
Next.js Frontend	Vercel API Gateway	POST /api/analyze	Starts scan.
FastAPI Backend	Neon PostgreSQL	SELECT / SQL	Gets laws.
FastAPI Backend	Z Ai GLM API	POST /v1/chat	Gets AI reasoning.
FastAPI Backend	Neon PostgreSQL	INSERT / SQL	Saves result.

f. System Interaction Logic



- ***Frontend ↔ Backend:*** *The Next.js client sends the image payload via POST /analyze. The backend acts as a secure orchestrator to manage image processing and API calls.*
- ***Backend ↔ Database:*** *The FastAPI Backend communicates with Neon PostgreSQL using SQL queries to fetch the latest regulatory rules and store final results. This ensures persistence of scan history for the user.*
- ***Backend ↔ External API (Reasoning):*** *The backend builds a specialized prompt and sends it to Z.AI GLM via HTTPS. This hides API keys from the frontend and allows the server to validate the AI's response before the user sees it.*

g. Distributed Components

Although the system is managed under a unified deployment, it is distributed across specialized cloud environments to ensure high availability:

- **Vercel Edge (Edge Layer):** The Next.js frontend is distributed globally via Vercel's Edge Network. This ensures low-latency UI rendering for users regardless of their physical location.
- **Vercel Functions (Serverless Compute Layer):** The FastAPI backend logic runs in a distributed serverless environment. Each request triggers an independent function instance, allowing the system to scale horizontally to handle multiple concurrent scans during peak periods.

© UMHackathon 2026

- **Neon Serverless Postgres (Storage Layer):** The database is decoupled from the compute layer. It utilizes a serverless storage architecture that scales storage and compute independently, ensuring data is always accessible by the distributed backend functions.
- **Z.AI GLM (AI Inference Layer):** The reasoning logic is offloaded to a distributed external API. This allows the system to leverage massive LLM compute power without requiring local GPU resources.

2.2.2. Sequence Diagram

a. ***User Interaction Flow / Customer Journey***

The primary journey follows a vendor who needs to verify if a new skincare product meets local safety regulations.

1. ***Entry:*** *The Vendor logs into the Indoroute AI dashboard.*
2. ***Action:*** *The Vendor uploads a high-resolution photo of the product's ingredient label.*
3. ***Processing:*** *The system extracts text, compares it against the database, and uses AI to reason through chemical synonyms.*
4. ***Outcome:*** *The Vendor receives a visual risk assessment and a downloadable compliance report.*

b. ***Step-by-Step Interactions***

The following sequence explains the technical "handshake" between components:

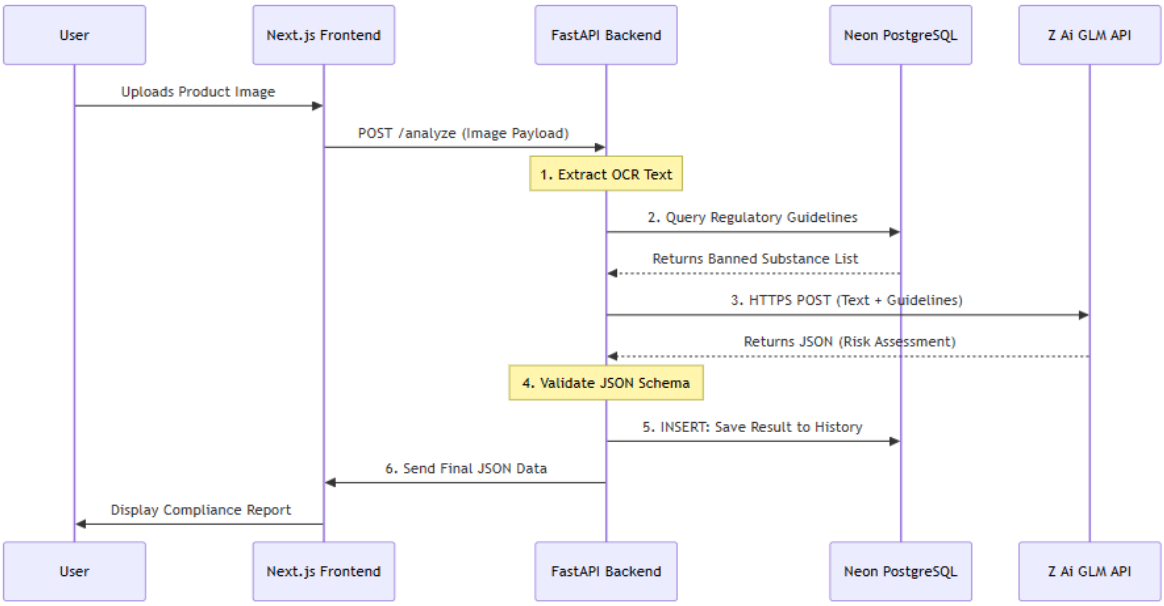
- ***Step 1:*** *The User uploads an image via the Next.js Frontend.*
- ***Step 2:*** *The Frontend sends the image to the FastAPI Backend via a secured Vercel Serverless Function.*
- ***Step 3:*** *The Backend queries Neon PostgreSQL to fetch the current "Banned & Restricted Substances" list.*
- ***Step 4:*** *The Backend sends the extracted label text + the regulatory data to the Z.AI GLM API.*
- ***Step 5:*** *Z.AI GLM performs multi-step reasoning to identify violations and returns a structured JSON object.*
- ***Step 6:*** *The Backend parses the JSON and performs a SQL INSERT to Neon PostgreSQL to save the result.*
- ***Step 7:*** *The Backend returns the final risk data to the Frontend, which updates the user's dashboard.*

c. ***Feature Operation (Start to Finish)***

This interaction demonstrates three key system features working in a single loop:

1. ***Automated Extraction:*** *Converts raw pixels into searchable chemical strings.*
2. ***AI-Driven Compliance Reasoning:*** *Evaluates complex ingredients against laws using LLM logic.*
3. ***Data Persistence:*** *Ensures all regulatory checks are logged for future audits.*

© UMHackathon 2026



2.2. Technological Stack

2.2.1. Frontend: *Next.js – Chosen for its seamless Vercel integration, fast Server-Side Rendering (SSR), and responsive design capabilities for mobile web users.*

2.2.2. Backend: *FastAPI – High-performance Python framework used to build serverless micro-agents; provides automatic OpenAPI documentation and high-speed JSON parsing.*

2.2.3. Database: *Neon PostgreSQL – A serverless relational database that allows for instant branching and scaling, ensuring user scan history and regulatory data are stored securely.*

2.2.4. Cloud/Deployment: *Vercel (Unified Hosting and Serverless Functions), providing a globally distributed edge network for the application.*

2.3. Key Data Flows

2.3.1. Data Movement and Storage

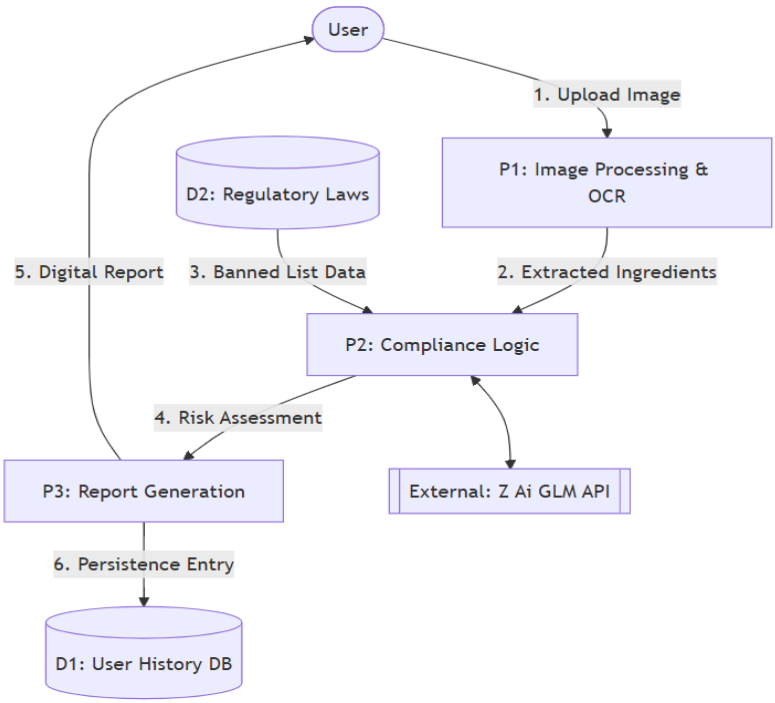
Data enters the system as an unstructured image and is progressively transformed into structured regulatory insights.

- ***Input:*** *Raw binary image data.*
- **Transformation:** Image -> OCR String -> JSON Object (via GLM) -> Relational Database Entry.
- **Storage:** Data is partitioned into User History (for tracking) and Regulatory Master Data (for grounding AI logic).

2.3.1.1. Data Flow Diagram (DFD)

© UMHackathon 2026

This Level 1 DFD illustrates how data moves across external entities, processes, and databases.

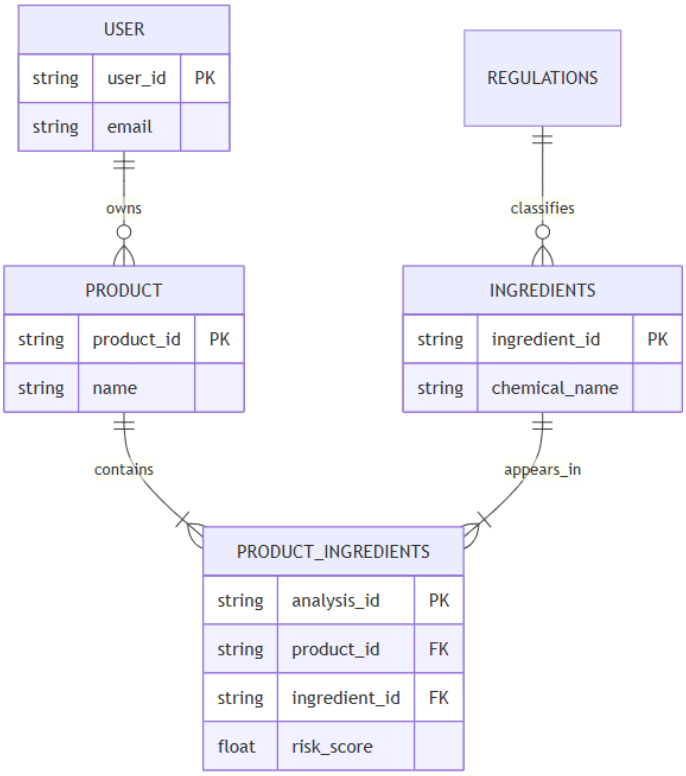


- ***External Entities:*** *User (SME Vendor) and Z Ai GLM API.*
- ***Processes:*** *P1 (Image Processing), P2 (Compliance Logic), and P3 (Report Generation).*
- ***Black Hole Compliance:*** *Every process has a defined output. For instance, P2 (Compliance Logic) consumes "Extracted Text" and "Regulatory Laws" to produce a "Risk Assessment," ensuring no data is lost in an isolated process.*

© UMHackathon 2026

2.3.2. Normalized Database Schema

The database is designed to Third Normal Form (3NF) to ensure data integrity and eliminate redundancy.



- **Normalization Logic:** We use a Junction Table (Product_Ingredients) to resolve the Many-to-Many relationship between Products and Ingredients. This ensures that a single ingredient name is stored only once in the master list, even if it appears in 1,000 different user scans, optimizing storage and search speed.

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of IndoRoute AI, focusing on a high-impact MVP to showcase technical feasibility during the hackathon.

3.1. Minimum Viable Product:

#	Features	Description
1	AI Ingredient Scanner	Users upload product label photos; system extracts chemical names using OCR and parses them into a searchable format.
2	Automated Compliance Check	GLM compares extracted ingredients against BPOM/NPRA databases stored in Neon DB to identify restricted substances.
3	Risk Scorecard	Generates a visual summary (Pass/Fail) with detailed reasoning for specific ingredient violations.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	Must handle concurrent scan requests during the demo.	Use Vercel Serverless Functions to auto-scale compute based on demand.
Reliability	99% uptime for the scan engine; no data loss during scans.	SQL transaction safety in Neon PostgreSQL ensures scan history is saved before returning results.
Maintainability	Easy to update regulatory rules without changing code.	Store BPOM/NPRA rules in a separate Database table for instant updates via the dashboard.

© UMHackathon 2026

Token Latency	*GLM API calls must return a response within 5s at p95.*	*Implement prompt optimization and limit the context window to essential regulatory snippets.*
Cost Efficiency	*Reduce average API token consumption per scan by 60% to 80% for repeat queries.*	*1. Exact string matching in Redis for known ingredients.* *2. Storing GLM analysis results in Neon DB to skip reasoning for identical products.*

3.3. Out of Scope / Future Enhancements

a. ***Direct BPOM Filing:*** *Automated submission of compliance results to government portals.*
b. ***Alternative Suggester:*** *AI suggesting safer chemical substitutes for banned ingredients.*
c. ***Offline Mode:*** *Localized OCR and basic compliance checking for areas with poor connectivity.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Grayscale Rollout & A/B Testing: New prompt templates for the GLM are released to a "Staging" environment on Vercel for team testing. Once validated, they are rolled out to 10% of users to monitor for "hallucinations" before a 100% production release.

4.1.2. ***Emergency Rollback (ER) & Golden Release***: If the GLM returns malformed JSON or invalid data, the system triggers an ER to the "Golden Version" (the last stable prompt/code version) to ensure the scan feature remains functional.

4.1.3. Priority Matrix:

4.1.3.1. P1 Critical: *If the Z Ai GLM API is unreachable, the system displays a "Maintenance Mode" notification and prevents new scans.*

4.1.3.2. P2 High: *If OCR accuracy drops below 80%, the system prompts the user to manually edit the extracted text before proceeding to AI analysis.*

4.2. Monitoring:

We utilize Vercel Runtime Logs and custom Sentry alerts to track system health. If API latency exceeds 10s or error rates hit 5%, an automated notification is triggered.

4.3. Assumptions:

a. Users have a stable internet connection to upload images and receive AI analysis.

© UMHackathon 2026

b. *Regulatory data provided by BPOM/NPRA can be digitized and stored in a relational format.*

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Core reasoning engine for compliance analysis.	High: Rate limits or downtime. Mitigation: Implement request caching.
Neon PostgreSQL	Serverless data storage for history and laws.	Low: Potential cold starts. Mitigation: Keep-alive pings.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days, executed as a high-velocity sprint from April 18 to April 26, 2026. The project follows a Phase-Gate model to ensure that infrastructure is stable before AI logic is integrated.

5.1. Project Timeline:

Day	Technical Milestones & Deliverables
1-2	• Monorepo setup • Neon DB provisioning • 3NF Schema migration • Seed master data (inventory.json).
3	• Z.AI GLM client setup • "Policy Sentinel" prompt drafting • Mock news alert ingestion logic.
4	• "Compliance Scanner" logic (BPOM rule matching) • Beacukai news scraper • Villain SKU data seeding.
5	• "Profit Shield" formula implementation • Logistics rates parser • Deployment to Vercel
6	• "Smart Router" trade-off reasoning • Cross-module integration • Feature Freeze at end of day.
7	• GLM latency optimization (<8s) • End-to-end bug triaging • Responsive UI/UX polish.

© UMHackathon 2026

8	• Live URL smoke testing • Pitch video recording • Final SAD/QATD documentation and screenshots.

5.2. Team Members Role:

- Lau Wei Zhong — Infra Lead — Backend architecture, Neon DB logic, FastAPI routers, and CI/CD pipelines.
- Tan Kang Zheng — *Data Engineer — Master JSON data, Beacukai news scraping, and logistics rate parsing.*
- Eugene Ngai Kai Qing — AI Engineer — Prompt engineering, structured JSON validation, and GLM fallback handling.
- Tan Jun Yong — Frontend Lead — Next.js App Router, Dashboard KPI visualization, and real-time alert UI.
- Lee Wuey Yong — Docs & QA — PRD/SAD/QATD documentation, 5×5 Risk Matrix, final integration and demo walkthrough.

5.3. Recommendations:

- ***Hybrid Logic:*** *Python handles all deterministic math for 100% accuracy; GLM provides human-readable reasoning.*
- ***Prompt Fallback:*** *"Golden Release" strategy; if GLM fails, system serves a cached stable response to prevent UI crashes.*
- ***Efficient Persistence:*** *Idempotent seeding (ON CONFLICT) and result caching to minimize DB load and token burn.*
- ***Infrastructure Tuning:*** *Neon DB branching for schema safety and manual "wake-up" pings for serverless performance.*

© UMHackathon 2026